Механизм: адресный перевод

При разработке виртуализации ЦП мы сосредоточимся на общем механизме, известном как **ограниченное прямое выполнение** (или LDE). Идея, лежащая в основе LDE проста: по большей части, пусть программа работает непосредственно на оборудовании; однако, в определенные ключевые моменты времени (например, когда процесс выдает системный вызов, или происходит прерывание таймера), ОС будет принимать участие и гарантировать, что “правильная” вещь происходит. Таким образом, ОС, с небольшой аппаратной поддержкой, старается изо всех сил не мешать работающей программе, чтобы обеспечить эффективную виртуализацию; однако, **вмешиваясь** в эти критические моменты времени, ОС гарантирует сохранность контроля над оборудованием. Эффективность и контроль вместе являются двумя основными целями любой современной операционной системы.

В виртуализации памяти мы будем следовать аналогичной стратегии, достигая как эффективности, так и контроля при обеспечении желаемой виртуализации. Эффективность диктует, чтобы мы использовали аппаратную поддержку, которая сначала будет довольно рудиментарной простой) (например, всего несколько регистров), но потом станет довольно сложной (например, TLBs, поддержка таблиц страниц и т. д.). Управление подразумевает, что ОС гарантирует, что никакое приложение не может получить доступ к любой памяти, кроме своей собственной; таким образом, чтобы защитить приложения друг от друга, и ОС от приложений, нам здесь тоже понадобится помощь от аппаратного обеспечения. Наконец, нам понадобится немного больше от системы VM с точки зрения гибкости; в частности, мы хотели бы, чтобы программы могли использовать свои адресные пространства любым способом, которым они хотели бы, тем самым упрощая программирование системы. И таким образом мы приходим к сути:

КАК ЭФФЕКТИВНО И ГИБКО ВИРТУАЛИЗИРОВАТЬ ПАМЯТЬ.
(Как построить эффективную виртуализацию памяти? Как мы обеспечиваем гибкость, необходимую для приложений? Как сохранить контроль над тем, к каким местам памяти приложение может получить доступ, и таким образом гарантировать, что доступ к памяти приложениями будет правильно ограничен? Как мы делаем все это эффективно?)

Общий метод, который мы будем использовать, называется **аппаратным преобразованием адресов** или просто **переводом адресов**. При преобразовании адресов оборудование преобразует каждый доступ к памяти (например, выборку, загрузку или сохранение инструкции), изменяя **виртуальный** адрес, предоставленный инструкцией, на **физический** адрес, где фактически находится требуемая информация. Таким образом, для каждой ссылки на память аппаратное обеспечение выполняет преобразование адресов, чтобы перенаправить ссылки на память приложения в их фактическое расположение в памяти.

Конечно, аппаратное обеспечение само по себе не может виртуализировать память, так как оно просто обеспечивает низкоуровневый механизм для этого. ОС должна принимать участие в ключевых моментах для настройки оборудования таким образом, чтобы выполнялись правильные переводы; таким образом, она должна **управлять памятью**, отслеживать, какие места свободны, а какие используются и поддерживать контроль над тем, как используется память.

Еще раз, цель всей этой работы - создать красивую **иллюзию**, что у программы есть своя частная память, в которой находится ее собственный код и данные. За этой виртуальной реальностью скрывается уродливая истина: многие программы фактически разделяют память одновременно, поскольку процессор (или процессоры) переключается между запуском одной программы и следующей. Благодаря виртуализации ОС (с помощью оборудования) превращает уродливую машинную реальность в полезную, мощную и простую в использовании абстракцию.

15.1 Предубеждения

Мы предположим, что адресное пространство пользователя должно быть размещено в физической памяти последовательно. Для простоты мы также предположим, что **размер адресного пространства не слишком велик; в частности, он меньше размера физической памяти.** Наконец, мы также предположим, что **каждое адресное пространство имеет одинаковый размер**. Не волнуйтесь, если эти предположения звучат нереалистично; мы будем реализировать их по мере продвижения, тем самым достигая реалистичной виртуализации памяти.

15.2 Пример

Чтобы лучше понять, что нужно сделать для реализации перевода адресов и зачем нужен такой механизм, рассмотрим простой пример. Представьте себе процесс, адресное пространство которого указано на рисунке 15.1. Здесь мы рассмотрим короткую последовательность кода

Интерпозиция - это универсальный и мощный метод, который часто используется для большей эффективности в компьютерных системах. При виртуализации памяти аппаратное обеспечение будет вмешиваться в каждый доступ к памяти и преобразовывать каждый виртуальный адрес, выданный процессом, в физический адрес, где фактически хранится требуемая информация. Тем не менее, общий метод интерпозиции гораздо более широко применим; Одним из преимуществ такого подхода является **прозрачность**;

который загружает значение из памяти, увеличивает его на три, а затем сохраняет значение обратно в память. Представление этого кода на языке C выглядит так:

```
void func() { int x = 3000;
    x = x + 3;           // нас интересует эта строчка кода
    ...
```

Компилятор превращает эту строку кода в ассемблерный код, который может выглядеть примерно так (в assembly x86). Используйте `objdump` в Linux или `otool` на Mac, чтобы дизассемблировать его:

```
128: movl 0x0(%ebx), %eax     ;load 0+ebx into eax
132: addl $0x03, %eax     ;add 3 to eax register 135:
movl %eax, 0x0(%ebx)      ;store eax back to mem
```

Этот фрагмент кода относительно простой; он предполагает, что адрес `x` был помещен в регистр `ebx`, а затем загружает значение по этому адресу в регистр общего назначения `eax` с помощью инструкции `movl`. Следующая инструкция добавляет 3 к `eax`, а последняя инструкция сохраняет значение из `eax` обратно в память на то же место.

На рисунке 15.1 (стр. 4) обратите внимание как код и данные размещены в адресном пространстве процесса; код из трех инструкций расположен по адресу 128 (в разделе кода вверху), а значение переменной `x` - по адресу 15 КБ (в стеке внизу). На рисунке начальное значение `x` равно 3000, его расположение находится в стеке.

Когда эти инструкции выполняются, с точки зрения процесса, происходят следующие обращения к памяти.

- Получить инструкцию по адресу 128
- Выполнить эту инструкцию (загрузить с адреса 15 КБ)
- Получить инструкцию по адресу 132
- Выполнить эту инструкцию (без ссылки на память)
- Получить инструкцию по адресу 135
- Выполнить эту инструкцию (сохранить по адресу 15 КБ)



Рисунок 15.1: **Процесс и его адресное пространство**

С точки зрения программы, ее адресное пространство начинается с адреса 0 и увеличивается максимум до 16 КБ; все создаваемые ею ссылки на память должны находиться в этих пределах. Однако для виртуализации памяти, ОС хочет поместить процесс в другое место физической памяти, не обязательно по адресу 0. Таким образом, возникает проблема: как **перенести** этот процесс в память **прозрачным** для процесса образом? Как мы можем создать иллюзию виртуального адресного пространства, начинающегося с 0, когда в действительности адресное пространство расположено по другому физическому адресу?

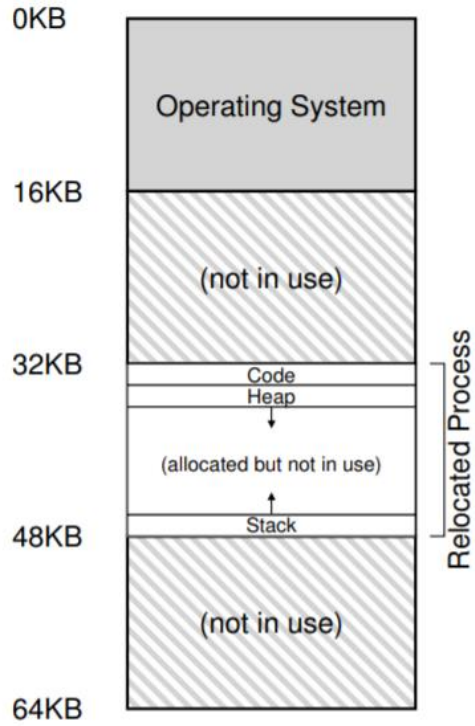


Рисунок 15.2: **Физическая память с одним перемещенным процессом**

Пример того, как может выглядеть физическая память после размещения адресного пространства процесса в памяти, приведен на рис. 15.2. На рисунке видно, что ОС использует первый слот физической памяти для себя, и что она переместила процесс из приведенного выше примера в слот, начиная с адреса физической памяти 32 КБ. Остальные два слота свободны (16 КБ-32 КБ и 48 КБ-64 КБ).

15.3 Динамическое (аппаратное) перемещение

Чтобы получить некоторое представление об аппаратном преобразовании адресов, мы сначала обсудим его первое воплощение. В конце 1950-х годов была введена простая идея, называемая **базой и границей**; этот метод также называется **динамическим перемещением**; мы будем использовать оба термина взаимозаменяемо.

В частности, нам понадобятся два аппаратных регистра в каждом процессоре: один называется **базовым** регистром, а другой - **границей** (иногда называемым регистром **предела**). Эта пара "база и границы" позволит нам разместить адресное пространство в любом месте физической памяти, и сделать это, гарантируя, что процесс может получить доступ только к своему собственному адресному пространству.

В этой настройке каждая программа записывается и компилируется так, как если бы она загружалась по нулевому адресу. Однако, когда программа начинает работать, ОС решает, где в физической памяти она должна быть загружена и устанавливает базовому регистру это значение. В приведенном выше примере ОС решает загрузить процесс по физическому адресу 32 КБ и таким образом устанавливает базовому регистру это значение.

Интересные вещи начинают происходить, когда процесс запущен. Теперь, когда любая ссылка на память генерируется процессом, она **преобразуется** процессором следующим образом: `физический адрес = виртуальный адрес + значение базового регистра`

ПЕРЕМЕЩЕНИЕ НА ОСНОВЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

До появления аппаратной поддержки, некоторые системы выполняли грубую форму перемещения исключительно с помощью программных методов. Основной метод называется **статическим перемещением**, в котором часть программного обеспечения, известная как **загрузчик**, берет исполняемый файл, который собирается быть выполненным, и переписывает его адреса к желаемому смещению в физической памяти.

Например, если инструкция была загружена из адреса 1000 в регистр (например, movl 1000, %eax), а адресное пространство программы было загружено начиная с адреса 3000 (а не 0, как думает программа), загрузчик перепишет инструкцию для смещения каждого адреса на 3000

(например, movl 4000, %eax). Таким образом достигается простое статическое перемещение адресного пространства процесса.

Однако статическое перемещение имеет множество проблем. Во-первых, и это самое главное, он не обеспечивает защиту, так как процессы могут генерировать плохие адреса и, таким образом, незаконно получить доступ к памяти другого процесса или даже ОС; в целом, аппаратная поддержка, вероятно, необходима для “истинной” защиты. Еще один минус заключается в том, что после размещения трудно впоследствии переместить адресное пространство в другое место.

Каждая ссылка на память, генерируемая процессом, является **виртуальным адресом**; аппаратные средства в свою очередь добавляют содержание Базового регистра к этому адресу, и результат является **физическим адресом**, который может быть выполнен к системе памяти.

Чтобы лучше понять это, давайте проследим, что происходит при выполнении одной инструкции. В частности, давайте рассмотрим одну инструкцию из нашей предыдущей последовательности:

```
128: movl 0x0(%ebx), %eax
```

Счетчик программы (PC) установлен на 128; когда оборудованию нужно получить эту инструкцию, оно сперва добавляет это значение к значению базового регистра 32 KB (32768) для того чтобы получить физический адрес 32896; оборудование после этого выбирает инструкцию из этого физического адреса. Затем процессор начинает выполнение инструкции. В какой-то момент процесс затем выдает нагрузку с виртуального адреса 15 КБ, который процессор берет и снова добавляет к значению базового регистра (32 КБ), получая конечный физический адрес 47 КБ и, таким образом, желаемое содержимое.

Преобразование виртуального адреса в физический - это именно тот метод, который мы называем **преобразованием адресов**; то есть оборудование принимает виртуальный адрес, на который процесс думает, что он ссылается, и преобразует его в физический адрес, на котором фактически находятся данные. Поскольку это перемещение адреса происходит во время выполнения, и поскольку мы можем перемещать адресные пространства даже после запуска процесса, этот метод часто называют **динамическим перемещением**.

Совет: Динамическое перемещение на основе аппаратных средств

Базовый регистр используется для преобразования виртуальных адресов (генерируемых программой) в физические адреса. Регистр **границы** (или **предела**) гарантирует, что такие адреса находятся в пределах адресного пространства. Вместе они обеспечивают простую и эффективную виртуализацию памяти.

Регистр bounds помогает в защите. В частности, процессор сначала проверит, что ссылка на память находится в `пределах границ`, чтобы убедиться, что она допустима; в примере приведенном выше регистр bounds всегда будет установлен в 16 КБ. Если процесс генерирует виртуальный адрес, который больше границ, или отрицателен, ЦП вызовет исключение, и процесс, скорее всего, будет завершен. Таким образом, смысл границ состоит в том, чтобы убедиться, что все адреса, генерируемые процессом, являются законными и находятся в «пределах» процесса.

Отметим, что регистры base (базовый регистр) и bounds (регистр границ) являются аппаратными структурами, хранящимися на чипе (одна пара на процессор). Иногда часть процессора, которая помогает с преобразованием адресов, называют **блоком управления памятью (MMU);** по мере

разработки более сложных методов управления памятью мы будем добавлять в MMU больше схем.

Небольшая информация о bounds регистрах, которые могут быть определены одним из двух способов. Один из способов - хранение размера адресного пространства, и, таким образом, аппаратное обеспечение проверяет по нему виртуальный адрес перед тем как добавить его в базу. Во втором случае он определяет физический адрес конца адресного пространства, и, таким образом, аппаратное обеспечение сначала добавляет его базу, а затем проверяет находится ли адрес в пределах границ. Оба метода логически эквивалентны; для простоты мы обычно используем первый метод.

Примеры переводов

Чтобы понять адресный перевод с помощью **базы** и **границы** более детально, давайте взглянем на пример. Представьте, что процесс с адресным пространством размером 4 КБ (да, нереально маленьким) был загружен по физическому адресу 16 КБ. Вот результаты нескольких переводов адресов:

Virtual Address		Physical Address
0	→	16 KB
1 KB	→	17 KB
3000	→	19384
4400	→	*Fault (out of bounds)*

Как вы можете видеть из примера, мы просто добавляем базовый адрес к виртуальному адресу (это можно рассматривать как смещение в адресном пространстве), чтобы получить физический адрес. Только если виртуальный адрес «слишком большой» или отрицательный, результатом будет ошибка, вызывающая исключение.

Структура данных.

ОС должна отслеживать, какие части свободной памяти не используются, чтобы иметь возможность выделять память процессам. Конечно, для такой задачи можно использовать множество различных структур данных; самый простой (который мы предложим здесь) - это **free list**, который просто представляет собой список диапазонов физической памяти, которые в настоящее время не используются.

15.4 Аппаратная поддержка: краткое изложение

Теперь давайте подытожим, что нам нужно из оборудования (также см. рис. 15.3, стр. 9). Во-первых, как описано в главе о виртуализации ЦП, нам требуются два разных режима ЦП. ОС работает в **привилегированном режиме** (или **режиме ядра**), где она имеет доступ ко всей машине; приложения работают в **пользовательском режиме**, где они ограничены в том, что они могут делать. Один бит, сохраненный в каком-то **значении состоянии процессора**, указывает, в каком режиме в настоящее время работает процессор; в определенных особых случаях (например, системный вызов, или какое-либо другое исключение, или прерывание) процессор переключает режимы.

Аппаратные средства должны также обеспечить **базовый регистр и регистр предела;** каждый ЦП таким образом имеет дополнительную пару регистров, часть **блока управления памятью (MMU)** ЦП. Когда пользовательская программа запущена, аппаратное обеспечение преобразует каждый адрес, добавляя базовое значение к виртуальному адресу, созданному пользовательской программой. Оборудование должно также быть в состоянии проверить, допустим ли адрес, который выполнен при помощи bound регистра и некоторых схем в ЦП.

Аппаратное обеспечение должно предоставлять специальные инструкции по изменению базовых и граничных регистров, позволяя ОС изменять их при запуске различных процессов. Эти инструкции **являются привилегированными**; только в режиме ядра (или привилегированном режиме) регистры могут быть изменены.

Наконец, ЦП должен быть в состоянии генерировать **исключения** в ситуациях, когда пользовательская программа пытается получить доступ к памяти незаконно (с адресом, который является «вне границ»); в этом случае ЦП должен остановить выполнение пользовательской программы и запустить **обработчик исключений ОС** (“out-of-bounds”). Обработчик ОС может затем выяснить, как ему реагировать, но как правило, он просто завершает этот процесс*. Аналогично, если пользовательская программа пытается изменить значения (привилегированных) регистров base и bounds, ЦП должен вызвать исключение и запустить обработчик. ЦП также должен предоставить метод, чтобы сообщить ему о местоположении этих обработчиков; таким образом, необходимы еще несколько привилегированных инструкций.

Требования к оборудованию	Заметки
Привилегированный режим	Требуется для предотвращения выполнения привилегированных операций процессами, работающими в пользовательском режиме
Base/bounds регистры	Требуется пара регистров на ЦП для поддержки преобразования адресов и проверки границ
Возможность перевода виртуальных адресов и проверки нахождения в пределах границ	Схемы для выполнения трансляций и проверки границ;
Привилегированные инструкции для обновление base/bounds	ОС должна иметь возможность устанавливать эти значения перед разрешением пользователю запускать программы
Привилегированные инструкции для регистрации обработчиков исключений	ОС должна быть в состоянии сказать аппаратному обеспечению какой код выполнять при возникновении исключения
Возможность создавать исключения	Когда процессы пытаются получить доступ к привилегированным инструкциям или внешней (не своей) памяти

Рис. 15.3: **Динамическое перемещение: требования к оборудованию**

15.5 Проблемы операционной системы

Всесте с тем, как оборудование начинает предоставлять новые функции для поддержки динамического перемещения, у ОС появляются новые проблемы, которые она должна обрабатывать; сочетание аппаратной поддержки и управления ОС приводит к реализации простой виртуальной памяти. В частности, есть несколько критических моментов, когда ОС должна принять участие в реализации нашей base-and-bounds версии виртуальной памяти.

Во-первых, ОС должна принимать участие при создании процесса, находя место для его адресного пространства в памяти. К счастью, учитывая наши предположения о том, что каждое адресное пространство (А) меньше размера физической памяти и (Б) имеет одинаковый размер, ОС может легко просматривать физическую память как массив слотов и отслеживать, свободен ли каждый из них или используется. При создании нового процесса ОС придется искать структуру данных (часто называемую **свободным списком**), чтобы найти место для нового адресного пространства, а затем пометить его как используемое. С адресными пространствами переменного размера дела обстоят немного сложнее, но мы оставим эту проблему для будущих глав.

Давайте рассмотрим пример. На рис. 15.2 видно, что ОС использует для себя первый слот физической памяти и что она переместила процесс из приведенного выше примера в слот, начиная с адреса физической памяти 32 КБ. Другие два слота свободны (16 КБ-32 КБ и 48 КБ, 64 КБ), таким образом, **свободный список** должен состоять из этих двух записей.

Во-вторых, ОС должна выполнять определенную работу, когда процесс завершен (т. е., когда он корректно завершает работу или принудительно убит из-за неправильного поведения), а именно освобождать выделенную раннее память для процесса*, чтобы потом использовать ее другими процессами или ОС. После завершения процесса ОС, таким образом, помещает эту* память обратно в **свободный список** (free list) и очищает любые связанные структуры данных по мере необходимости.

В-третьих, ОС должна также выполнить несколько дополнительных дейсвтий при переключении контекста. В конце концов, на каждом процессоре есть только одна пара базовых и граничных регистров, и их значения различаются для каждой запущенной программы, так как каждая программа загружается по другому физическому адресу в памяти. Таким образом, ОС должна сохранить и восстановить значения пары регистров "основание-граница" при переключении

Требования к ОС	**Заметки**
Управление памятью	Необходимость выделения памяти новым процессам; Высвобождение памяти завершенных процессов; Управление памятью через ***free list***
Base/bounds управление	Необходимо установить правильное значение *base/bounds* регистров ри переключении контекста

Обработка исключений	Код, который нужно запустить, когда возникают исключения; вероятное действие - завершить процесс-нарушитель

Рисунок 15.4**: Динамическое перемещение: обязанности операционной системы**

между процессами. В частности, когда ОС решает остановить выполнение процесса, она должна сохранить значения base и bounds регистров в памяти в некоторой структуре каждого процесса, такой как **структура процесса** или **блок управления процессом** (PCB – process control block). Точно так же, когда ОС возобновляет запущенный процесс (или запускает его в первый раз), она должна установить правильные значения base и bounds на процессоре для этого процесса.

Следует отметить, что при остановке процесса (т. е. когда он не запущен) ОС может довольно легко переместить адресное пространство из одного места в памяти в другое. Для того, чтобы переместить адресное пространство процесса, ОС сначала откладывает процесс; затем, ОС копирует адресное пространство из текущего местоположения в новое; и, наконец, ОС обновляет сохраненный базовый регистр (в структуре процесса), чтобы указать новое местоположение. Когда процесс возобновляется, его (новый) базовый регистр восстанавливается, и он начинает работать снова, не обращая внимания на то, что его инструкции и данные теперь находятся в совершенно новом месте в памяти.

В-четвертых, ОС должна предоставлять **обработчики исключений** или вызываемые функции, как описано выше; ОС устанавливает эти обработчики во время загрузки (с помощью привилегированных инструкций). Например, если процесс пытается получить доступ к памяти за пределами своих границ, ЦП вызовет исключение; ОС должна быть готова действовать при возникновении такого исключения. Общей реакцией ОС будет враждебность: она, скорее всего, прекратит враждебный процесс. ОС должна быть хорошо защищена от машины, на которой она работает, и, в следствие из этого, она отрицательно относится к процессу, пытающемуся получить доступ к памяти или выполнить инструкции, которые он не должен.

Рисунок 15.5 иллюстрирует большую часть взаимодействий оборудования/ОС на временной шкале. На рисунке показано, что делает ОС во время загрузки, чтобы подготовить машину к использованию, а затем, что происходит, когда процесс (процесс A) начинает работать; обратите внимание, как его преобразования памяти обрабатываются аппаратным обеспечением без вмешательства ОС. В какой-то момент происходит прерывание таймера, и ОС переключается на процесс B, который выполняет «плохую нагрузку» (на недопустимый адрес памяти); в этот момент ОС должна принять меры, завершив процесс и очистив память B, и удалив ее запись из таблицы процессов. Как видно из диаграммы, мы по-прежнему придерживаемся базового подхода **ограничения прямого исполнения**. В большинстве случаев ОС просто настраивает оборудование соответствующим образом и позволяет процессу работать непосредственно на ЦП; только тогда, когда процесс плохо себя ведет, ОС должна участвовать.

OS @ boot (kernel mode)	Hardware	
initialize trap table		
	remember addresses of...	
	system call handler	
	timer handler	
	illegal mem-access handler	
	illegal instruction handler	
start interrupt timer		
	start timer; interrupt after X ms	
initialize process table		
initialize free list		

OS @ run (kernel mode)	Hardware	Program (user mode)
To start process A:		
allocate entry in process table		
allocate memory for process		
set base/bounds registers		
return-from-trap (into A)		
	restore registers of A	
	move to **user mode**	
	jump to A's (initial) PC	
		Process A runs
		Fetch instruction
	Translate virtual address	
	and perform fetch	
		Execute instruction
	If explicit load/store:	
	Ensure address is in-bounds;	
	Translate virtual address	
	and perform load/store	
		...
	Timer interrupt	
	move to **kernel mode**	
	Jump to interrupt handler	
Handle the trap		
Call `switch()` routine		
save regs(A) to proc-struct(A)		
(including base/bounds)		
restore regs(B) from proc-struct(B)		
(including base/bounds)		
return-from-trap (into B)		
	restore registers of B	
	move to **user mode**	
	jump to B's PC	
		Process B runs
		Execute bad load
	Load is out-of-bounds;	
	move to **kernel mode**	
	jump to trap handler	
Handle the trap		
Decide to terminate process B		
de-allocate B's memory		
free B's entry in process table		

Рис. 15.5: **Ограниченный протокол прямого исполнения (динамическое перемещение).**

15.6 Краткое изложение

В этой главе мы расширили понятие ограниченного прямого выполнения с помощью специального механизма, используемого в виртуальной памяти, известного как **преобразование адресов**. С помощью преобразования адресов ОС может контролировать каждый доступ к памяти из процесса, обеспечивая доступ в пределах адресного пространства. Ключом к эффективности этого метода является аппаратная поддержка, которая быстро выполняет перевод для каждого доступа, превращая виртуальные адреса (как видит память сам процесс) в физические (фактический вид). Все это выполняется способом, который понятен для перемещенного процесса; процесс не имеет ни малейшего представления о том, что ссылки на память могут изменяться, это все создаёт замечательную иллюзию.

Мы также рассмотрели конкретную форму виртуализации, известную как база и границы или динамическое перемещение. Виртуализация Base-and-bounds довольно эффективна, поскольку для добавления базового регистра к виртуальному адресу требуется лишь немного больше аппаратной логики и проверка того, что адрес, сгенерированный процессом, находится в границах. Base-and-bounds также предполагает защиту; ОС и оборудование объединяются, чтобы гарантировать, что никакой процесс не может генерировать ссылки на память вне его собственного адресного пространства. Защита, безусловно, является одной из важнейших целей ОС; без него ОС не могла бы управлять машиной (если бы процессы могли свободно перезаписывать память, они могли бы легко делать неприятные вещи, такие как перезапись таблицы ловушек (trap table) и взять на себя систему).

К сожалению, этот простой метод динамического перемещения имеет свои недостатки. Например, как видно на рис.15.2, перемещенный процесс использует физическую память от 32 КБ до 48 КБ; однако, поскольку стек и куча процессов не слишком велики, все пространство между ними просто тратится впустую. Этот тип “отходов” обычно называют **внутренней фрагментацией**, так как пространство внутри выделенного блока используется не полностью (т. е. фрагментируется) и, таким образом, тратится впустую. Таким образом, нам понадобится более сложный механизм, чтобы лучше использовать физическую память и избежать внутренней фрагментации. Наша первая попытка будет небольшим обобщением базы и границ, известных как **сегментация**, о которой мы поговорим далее.